



LlamaTrade

ENGINEERING REFERENCE

Architecture & System Design

How every part of the platform works, end to end — services and their boundaries, the strategy DSL and compiler, the event-sourced portfolio ledger, data & multi-tenancy, durability, the event fabric, and observability. The single technical reference for the engineering team.

STATUS

LIVING DOC

DATE

June 16, 2026

AUDIENCE

Engineering

SCOPE

Full system, end to end

Contents

01 System at a Glance

product loop · microservices · communication model

02 Service Topology

the service map · ports · responsibilities · dependencies

03 Communication & Protocols

Connect vs gRPC · Redis · proto source of truth · no gateway

04 End-to-End Flows

strategy lifecycle · live runner · market data · backtest

05 Strategy DSL & Compiler

grammar · indicators · weight methods · compilation

06 The Portfolio Ledger

sleeves · lots · invariants · reconciliation · block-and-allocate

07 Data Model

tenant boundary · ledger cluster · shared bars

08 Multi-Tenancy & Security

JWT · RLS · RBAC · encryption · audit

09 Durability & State Authority

layered authority · idempotent orders · crash recovery

10 Event Fabric

Redis Streams · two delivery modes · idempotency · libs/events

11 Observability

three signals · cardinality rules · metric catalog · the money path

12 Stack & Deployment

tech stack · GKE · CI/CD · runtime

01 System at a Glance

LlamaTrade is a multi-tenant SaaS for algorithmic trading. Users build (or pick) strategies in a visual builder, backtest them against historical data, and run them live or in paper mode against **their own Alpaca brokerage account**. We never custody funds — Alpaca is the broker of record; we are the automation layer.

THE PRODUCT LOOP — EVERY FEATURE SERVES ONE CYCLE

01

Build / Pick

visual builder · template · AI
copilot → DSL

02

Backtest

vs SPY · Sharpe · drawdown ·
win rate

03

Deploy Paper

simulated money on live data

04

Go Live

real money · risk guardrails ·
circuit breaker

iterate ◀ tweak → re-test → redeploy — every run pins an immutable strategy version

The shape of the system

It is a set of **focused microservices**. The browser talks **directly** to each service over the Connect protocol (no API gateway); services talk to each other over gRPC; and asynchronous / real-time work flows through Redis. One service — **Portfolio** — owns the event-sourced **ledger** that is the book of record for money and positions.

- **10 services** (9 backend + web), ports `88xx`
- **Connect** (HTTP/1.1+JSON) browser→service; **gRPC** (HTTP/2) service→service
- **Redis** for Celery jobs, live streams, and the ledger fill bus
- **Postgres 16 + TimescaleDB** (bars); per-tenant **Row-Level Security**
- **Proto** is the single source of truth for contracts & enums
- **BYO Alpaca keys**, encrypted per tenant (AES-256-GCM)

Core design principles

The ledger is the book of record

Event-sourced, double-entry; per-sleeve positions, cash & P&L are pure folds. Conservation is a checksum, so drift is never silent.

Proto is the single source of truth

Contracts & enums defined once, generated for Python + TypeScript. Integers internally; strings only at external boundaries.

Durability layered by authority

The broker owns live positions, the ledger owns money & P&L, the executor owns order intent — no single session log to keep in sync.

Idempotent by construction

A deterministic `client_order_id` gives exactly-once submission and doubles as the ledger's attribution key on every fill.

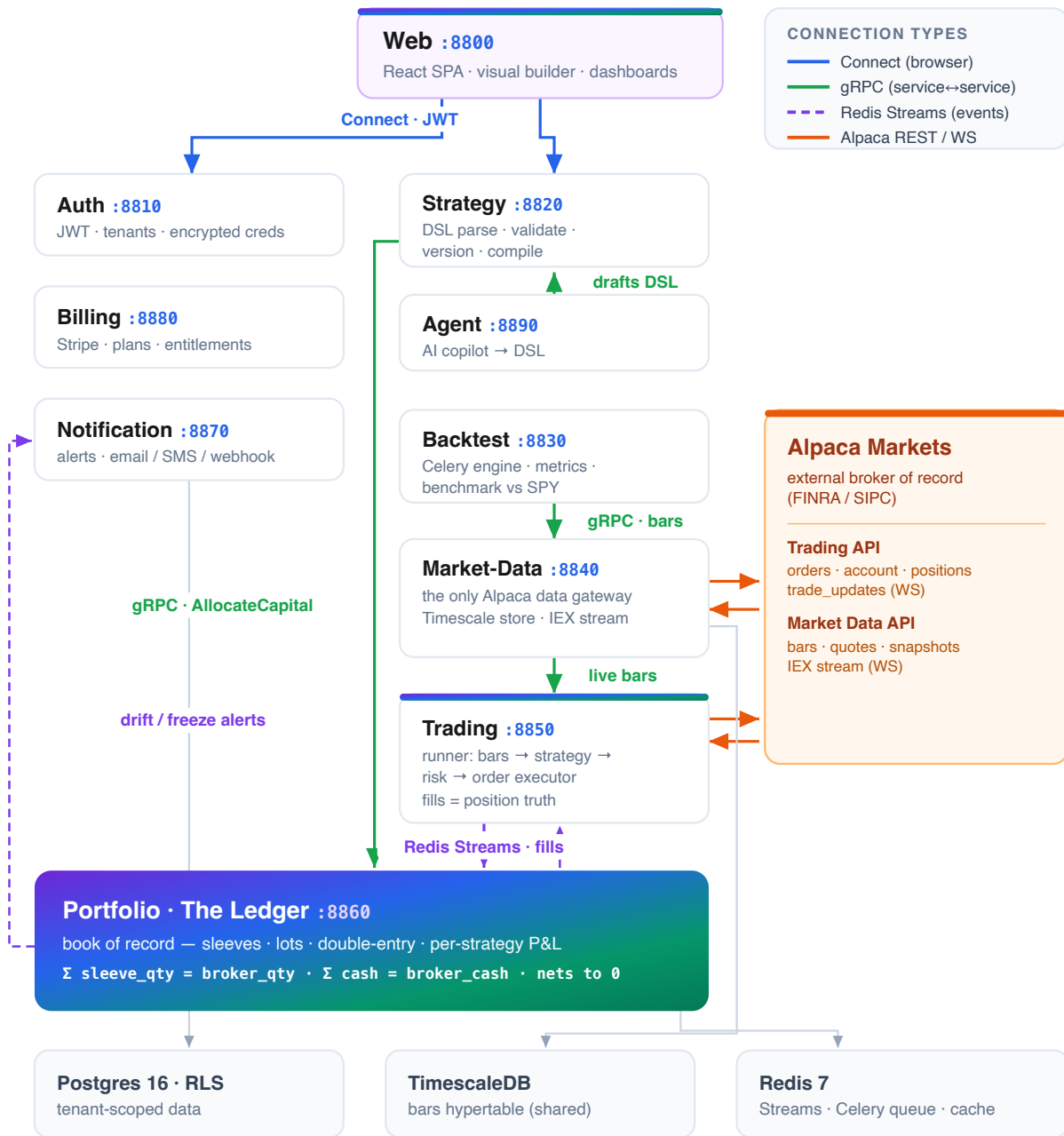
Isolated by default

`tenant_id` on every row, Row-Level Security as a backup, and `X-Tenant-ID` propagated across services.

Bring your own brokerage

Users connect their own Alpaca account; LlamaTrade automates it but never custodies funds — Alpaca is the broker of record.

The System in Action



Everything flows downhill — **Connect** from the browser, **gRPC** between services, **Alpaca** for execution, and **Redis-Streams** fills into the ledger, the double-entry book of record.

02 Service Topology

The browser talks to every service directly over **Connect** (a JWT per request); services call each other over **gRPC**; and **Portfolio** owns the event-sourced ledger — the book of record. The full flow map is on the previous page (*The System in Action*); grouped by role:

- Build** Strategy · Agent (AI copilot → DSL)
- Test & data** Backtest · Market-Data — the only Alpaca data gateway
- Execute** Trading — runner · risk + circuit breaker · orders ⇔ Alpaca
- Identity / ops** Auth · Billing · Notification
- Book of record** ★ Portfolio — the ledger (sleeves · double-entry · Σ sleeve = broker)
- Stores** Postgres 16 (RLS) · TimescaleDB (bars) · Redis 7 (queues · fills · progress)

SERVICE	PORT	RESPONSIBILITY	KEY DEPENDENCIES
Web	8800	React SPA — builder, dashboards (nginx/CDN)	all services (Connect)
Auth	8810	Users, tenants, roles, API keys, JWT, encrypted Alpaca creds	Postgres
Strategy	8820	Strategy CRUD, versioning, compile/validate, templates	Postgres · dsl · compiler
Backtest	8830	Historical simulation, metrics, benchmark comparison	Celery/Redis · Market Data · compiler
Market Data	8840	Historical bars (stored) + real-time Alpaca streaming; only data gateway	Alpaca · TimescaleDB · Redis
Trading	8850	Live/paper execution, risk, position tracking; emits fills to ledger	Alpaca (WS+REST) · Redis · Portfolio
Portfolio	8860	The ledger — book of record; positions, P&L, performance	Postgres · Redis · Market Data
Notification	8870	Alerts, webhooks, email/SMS/Slack	Postgres · SMTP/Twilio/Slack
Billing	8880	Subscriptions, plan limits, usage (Stripe); webhooks on :8881	Postgres · Stripe
Agent	8890	AI copilot — natural-language strategy building	LLM · Strategy · Backtest

03 Communication & Protocols

Three transport lanes, each for a job. Contracts are shared via **Protocol Buffers**; `make proto` generates Python and TypeScript from `libs/proto`.

browser → service

Connect

HTTP/1.1 + JSON · Bearer JWT validated by each service's shared middleware · visible in DevTools · native streaming

service → service

gRPC

HTTP/2 + binary protobuf · e.g.
Backtest→Market Data, Trading→Portfolio · X-Tenant-ID propagated

async / real-time

Redis

Celery backtests · live price & order streams · the ledger fill bus (Streams)

Proto is the source of truth

Enums are defined once in proto and generated for both languages — integers internally and for storage, strings only at external boundaries (Alpaca, display).

ENUMS — INTEGERS INTERNALLY, STRINGS ONLY AT THE EDGE

```
from llamatrade_proto.generated import ORDER_SIDE_BUY, ORDER_STATUS_FILLED
from src.models import order_side_to_str          # int → "buy"/"sell" for Alpaca

if order.side == ORDER_SIDE_BUY:                  # integers internally
    await alpaca.submit_order(side=order_side_to_str(order.side)) # strings only at the boundary
```

Why no API gateway decided

LlamaTrade uses **direct service communication** via Connect rather than a gateway. The rationale, for a frontend-only API with <15 services and strong shared-library discipline:

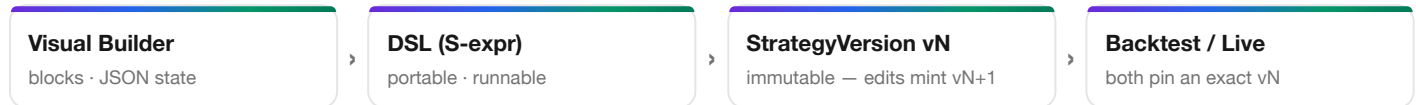
- **No SPOF** — a gateway down means all APIs down; per-service auth failures stay isolated.
- **Lower latency** — no proxy hop (+1–5ms), important for real-time data.
- **Native streaming** — Connect streams without gRPC-Web transcoding.
- **Independent scaling** — each service scales to its own load.
- **Simpler local dev** — no gateway process to run.
- **Consistency** via `llamatrade_common` shared middleware (auth, logging, errors).

Revisit if we expose a public third-party API, need per-key quotas, or grow past ~20 services — services already own their auth, so a gateway would layer on rather than replace.

04 End-to-End Flows

4.1 — Strategy lifecycle (immutable versions)

The visual builder compiles to a portable **DSL** (S-expression). Saving mints an **immutable StrategyVersion** (visual **config** JSON + compiled **sexpr**); edits always create vN+1. Every backtest and live session pins an exact version, so runs are reproducible.



4.2 — Live trading runner (two clocks, four loops)

Deploying a strategy starts a **runner** bound to the tenant's Alpaca credentials. It ticks on a **1-minute bar clock**, but a **date-based rebalance gate** (the **:rebalance** frequency, $\leq$ once/day) decides when target weights recompute.

THE RUNNER LOOP — FILLS ARE THE SOURCE OF TRUTH FOR POSITIONS

- 1 Data in** **1-min bars**
Runner consumes the live bar stream from Market Data and ticks every minute the market is open.
- 2 Strategy logic** **rebalance gate**
On a qualifying bar (gate open $\leq$ once/day) the compiled strategy recomputes target weights for its sleeve; otherwise it holds.
- 3 Risk + circuit breaker**
Sleeve-aware risk checks (buying power, position/order/daily-loss limits, rate); consecutive-loss / drawdown circuit breaker can halt the session.
- 4 Order out** **deterministic id**
One rebalance can emit many orders (one per symbol whose holding changes). Executor submits to Alpaca immediately (market/limit/stop/stop-limit/bracket) with a deterministic **client_order_id** for exactly-once submission.
- 5 Fills in** **trade_updates**
A **trade_updates** WebSocket delivers fills asynchronously — submission $\neq$ instant fill. Fills are the source of truth for positions, not optimistic local state.
- 6 Reconcile + book**
Every ~5 min the runner reconciles local positions against the broker and corrects drift; terminal fills are published to the portfolio ledger for per-sleeve P&L.

4.3 — Market data & backtest

Market Data is the single gateway to Alpaca and the system of record for price history. Historical bars live in a shared **TimescaleDB hypertable**; real-time trades/quotes/bars stream from Alpaca's WebSocket and fan out to clients, hottest values cached in Redis. *No other service calls Alpaca for data directly.*

Backtest fetches bars from Market Data over gRPC and runs the compiled strategy bar-by-bar with slippage and commission. It computes return, Sharpe, Sortino, max drawdown, win rate, profit factor and an equity curve — vs a benchmark (**SPY default**) for alpha/beta. Jobs run on **Celery**; progress streams to the UI.

4.4 — The Alpaca surface (everything via `libs/alpaca`)

Every Alpaca call — REST and WebSocket — goes through the shared `llamatrade_alpaca` library; no service talks to Alpaca directly or loads credentials itself. `paper=True` selects the paper environment (`paper-api.alpaca.markets` vs `api.alpaca.markets` ; market data from `data.alpaca.markets`).

Trading REST · `TradingClient`

<code>get_account()</code>	equity · buying power · cash → risk checks
<code>submit_order()</code>	place an order (types below) with a deterministic <code>client_order_id</code>
<code>cancel_order()</code> · <code>get_order()</code>	cancel · status (<code>filled_qty</code> , avg price)
<code>get_positions()</code> · <code>close_position()</code>	broker positions · flatten a symbol
<code>get_clock()</code>	market open/close · next session
<code>get_order_by_client_id()</code>	crash recovery — collapse a retry onto the same order

Market Data REST · `MarketDataClient`

<code>get_bars()</code> · <code>get_multi_bars()</code>	OHLCV — 1Min·5Min·15Min·1H·1D · store-first, Alpaca gap-fill
<code>get_latest_quote()</code> · <code>_trade()</code>	top of book · last print
<code>get_snapshot()</code> · <code>get_multi_snapshots()</code>	bar + quote + trade, combined
Served from the TimescaleDB store first; Alpaca only fills gaps. Redis TTLs: historical 24h · today 5m · quote 10s · snapshot 15s .	

Order types & time-in-force

ORDER TYPE	BEHAVIOR
market	immediate at best available price
limit	at <code>limit_price</code> or better
stop	becomes a market order when <code>stop_price</code> hits
stop_limit	becomes a limit order at the stop
trailing_stop	stop trails by <code>trail_price</code> / <code>trail_percent</code>
bracket / OCO	entry + attached stop-loss + take-profit

TIME-IN-FORCE	BEHAVIOR
day	Cancels at the session close
gtc	good-til-cancelled
ioc	fill what's available now, cancel the rest
fok	fill the whole order at once or cancel
	fills arrive async on <code>trade_updates</code> — submission ≠ fill (§4.5)

4.5 — Real-time streams & resilience

WebSocket streams

- `MarketDataStreamClient` — IEX trades/quotes/bars; **callback** delivery (`set_callbacks()` + `run()`); singleton with ref-counted subscriptions.
- `TradingStreamClient` — `trade_updates` order/fill events; **async-generator** (`async for event in stream()`). **Fills are the source of truth for positions.**
- `MockTradeStream` — in-memory trade stream for tests.

Resilience & credentials

- **REST** (`AlpacaClientBase`): token-bucket rate limiter · circuit breaker · retry w/ exponential backoff + jitter · Prometheus per-endpoint metrics.
- **WebSocket** (`AlpacaWebSocketBase`): shared transport · auth · reconnect w/ backoff + jitter.
- **Credentials** resolve: explicit `AlpacaCredentials` → per-tenant (decrypted from DB) → `ALPACA_API_KEY/SECRET` env fallback.
- **Singletons**: `get_trading_client() / _async()` · `get_market_data_client() / _async()` · `get_market_data_stream()` .

05 Strategy DSL & Compiler

The DSL is an **allocation language**, not a signal language — a strategy computes **target portfolio weights** (e.g. `{VTI: 60%, BND: 40%}`), never share counts. It is an S-expression with a lossless round-trip between text, AST, JSON, and visual blocks.

```
EXAMPLE — SECTOR ROTATION (REGIME FILTER + MOMENTUM TOP-N)

(strategy "Sector Rotation"
  :rebalance monthly :benchmark SPY
  (if (> (price SPY) (sma SPY 200)) ; bull regime?
    (filter :by momentum :select (top 3) :lookback 90 ; top 3 sectors
      (weight :method momentum :lookback 90
        (asset XLK) (asset XLF) (asset XLE) (asset XLV) (asset XLI)))
    (else (weight :method equal (asset BND) (asset GLD))))) ; defensive
```

Six block types

BLOCK	ROLE	FORM
strategy	Root container (exactly one); name, <code>:rebalance</code> , <code>:benchmark</code>	(strategy "..." :rebalance ... children)
asset	Leaf — the only construct that holds capital	(asset SYM :weight n)
weight	Allocation engine — assigns each child a fraction by <code>:method</code>	(weight :method m :lookback d :top n ...)
group	Transparent named container (readability/UI)	(group "..." :weight n ...)
if / else	Conditional allocation — branch on a market condition	(if <cond> then (else else))
filter	Rank candidates by a metric, keep top/bottom N before weighting	(filter :by c :select (top n) ...)

Weight methods

METHOD	STATUS
specified	full manual %
equal	full 100/N
momentum	partial equal-weights winners
inverse-volatility	full 1/σ
risk-parity	full ERC (covariance)
min-variance	full Σ ⁻¹ ·1 long-only
market-cap	rejected no fundamentals

Roadmap methods: volatility-targeting · HRP · dual-momentum · score-proportional momentum.

Indicators (16+)

- **Trend:** SMA, EMA, MACD, ADX
- **Momentum:** RSI, Stochastic, CCI, Williams %R, MFI, Momentum
- **Volatility:** Bollinger, ATR, Keltner, StdDev
- **Volume/channel:** OBV, VWAP, Donchian

14 of ~24 cross-platform table-stakes shipped. **Roadmap (researched):** ROC, Stochastic RSI, Aroon, Parabolic SAR, SuperTrend, Ichimoku, Hull/KAMA, A/D · CMF, pivot points, 52-wk-high, Z-score-vs-MA.

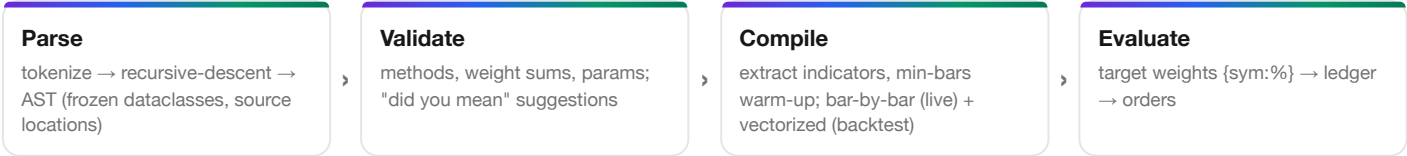
Conditions

- **Comparison** `> < >= <= = !=` — true every bar it holds
- **Crossover** `crosses-above/below` — true only on the transition (needs 2 bars)
- **Logical** `and / or / not`

TWO CLOCKS – THE BEHAVIOR-DEFINING GATE

Conditions are **evaluated** every bar but only **acted on** at the `:rebalance` cadence (`daily` ... `annually` ; finest grain is daily). A `crosses-above` that fires between rebalance dates of a `monthly` strategy is **missed** — prefer state comparisons (`>`) for low-frequency strategies.

Compilation pipeline



Stored on `StrategyVersion` in both forms: `config_sexpr` (canonical text) and `config_json` (compiled IR), plus extracted `symbols` / `timeframe` . The same compiled `StrategySession` drives live and backtest identically — no drift.

Rebalance frequencies & metrics

FREQUENCY	RECOMPUTES TARGET WEIGHTS
daily	first qualifying bar each trading day
weekly	first trading day of the week
monthly	first trading day of the month
quarterly	first trading day of the quarter
annually	first trading day of the year

Finest grain is `daily` — no sub-daily rebalancing today, even though 1-minute bars are ingested.

Value expressions — the operands of conditions

- **Numeric** — `50`, `0.30`, `-2.5` (no `%` — compare RSI to `30`, not `30%`).
- **Price** — `(price SYM [:close|:open|:high|:low|:volume])`; the field is the keyword.
- **Indicator** — `(name SYM params [:output])`, e.g. `(macd SPY 12 26 9 :signal)`.
- **Metric** — `(drawdown SYM) · (return SYM n) · (volatility SYM [n])`.

Metrics usable in any condition: `(drawdown SYM) · (return SYM n) · (volatility SYM [n])` — e.g. de-risk when `(> (drawdown SPY) 0.15)`.

EXAMPLE — RISK-BALANCED CORE (ALL-WEATHER)

```
(strategy "All-Weather Core"
  :rebalance quarterly :benchmark SPY
  (weight :method inverse-volatility :lookback 60
    (asset SPY) (asset TLT)
    (asset GLD) (asset DBC)))
```

How indicators shape weights — three mechanisms

The DSL is allocation, not signals: an indicator can only influence the target weights in one of three ways.

branch

if / else

a condition flips the subtree between two allocations (regime switch)

rank

filter

`:by` a metric → keep `(top N)` / `(bottom N)` candidates before weighting

weight

weight :method

the method computes weights from price history (momentum, inv-vol...)

Example gallery — the language's range

STATIC — CLASSIC 60/40

```
(strategy "Classic 60/40"
  :rebalance quarterly :benchmark SPY
  (weight :method specified
    (asset VTI :weight 60)
    (asset BND :weight 40)))
```

TACTICAL — TREND REGIME SWITCH

```
(strategy "Trend Regime"
  :rebalance daily :benchmark SPY
  (if (> (sma SPY 50) (sma SPY 200))
    (weight :method specified
      (asset SPY :weight 60) (asset QQQ :weight 40))
    (else (weight :method specified
      (asset TLT :weight 50) (asset GLD :weight 50)))))
```

MEAN-REVERSION — NESTED RSI REGIMES

```
(strategy "RSI Mean Reversion"
:rebalance daily :benchmark SPY
(if (< (rsi SPY 14) 30)
(asset SPY :weight 100)
else (if (> (rsi SPY 14) 70)
(asset TLT :weight 100)
else (weight :method equal
(asset SPY) (asset TLT))))))
```

CONFIRMATION — MULTI-CONDITION CROSSOVER

```
(strategy "Confirmed Trend"
:rebalance daily :benchmark SPY
(if (and (crosses-above (sma SPY 50) (sma SPY 200))
(< (rsi SPY 14) 70)
(not (> (drawdown SPY) 0.10)))
(weight :method specified
(asset SPY :weight 70) (asset QQQ :weight 30))
else (asset BIL :weight 100)))
```

Authoring tips

- **Start static, then add logic** — begin with `specified` / `equal`, layer in `if` / `filter` once it works.
- **Match cadence to logic** — comparisons for low-frequency rebalances; crossovers shine on `daily`.
- **Always set a `:benchmark`** so reporting can compute alpha/beta.
- **Mind the warm-up** — `(sma SPY 200)` needs ≥ 200 bars before it produces signals.
- **Diversify the universe** for dynamic methods — momentum / inverse-vol need several uncorrelated candidates.
- **Target weights are per-sleeve**, not your whole account — see §06.

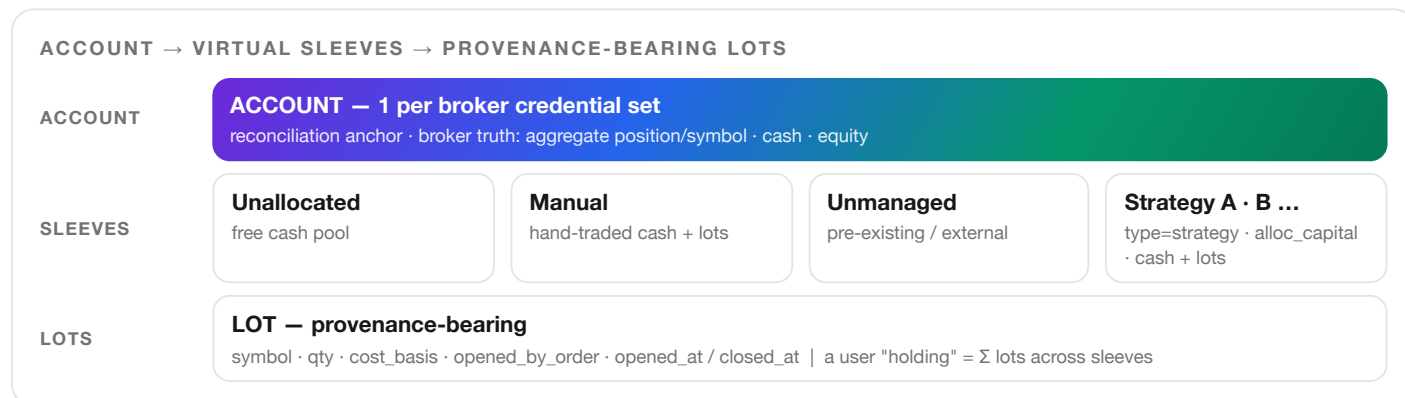
Built-in strategy families — what the DSL expresses

The platform ships templates for the allocation-expressible families below. Shorts, options, and pairs need execution features beyond today's long-only allocation model (roadmap).

FAMILY	IDEA	DSL BUILDING BLOCKS	STATUS
MA crossover	golden / death cross → risk-on vs risk-off	<code>if · sma/ema · crosses-above</code>	ships
Tactical regime (Faber GTAA)	hold above the 200-day, else cash/bonds	<code>if · sma · weight equal</code>	ships
RSI mean reversion	buy oversold, trim overbought	<code>if · rsi (nested)</code>	ships
Sector momentum rotation	hold the top-N sectors by trailing return	<code>filter :by momentum · weight momentum</code>	ships
Dual momentum (GEM)	absolute + relative momentum → winner or bonds	<code>if · return · filter</code>	ships
Risk parity / all-weather	ERC / inverse-vol across stocks/bonds/gold/commodities	<code>weight :method risk-parity</code>	ships
MACD trend	MACD line crossing its signal	<code>if · macd :signal · crosses-above</code>	ships
Donchian / Turtle breakout	20-day channel breakout, ATR-sized	<code>if · donchian (+ ATR sizing)</code>	partial
Volatility-managed	scale equity by realized vs target vol	needs a <code>vol-target</code> method	roadmap
Pairs / stat-arb	cointegrated spread z-score (long/short)	needs shorts + spread legs	roadmap
Options income	covered call · iron condor	needs options orders	roadmap

06 The Portfolio Ledger

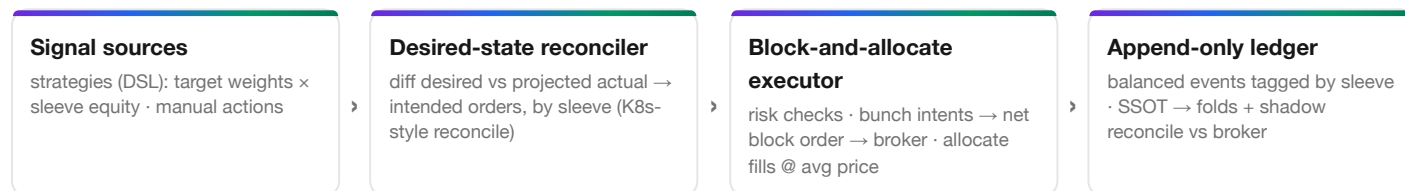
The hard problem: one Alpaca account serves **multiple strategies + manual trades**, but the broker only knows aggregates (one position per symbol, one cash balance). The ledger gives every holding **provenance**, prevents strategies from **interfering**, and accounts each strategy's capital correctly — via an **event-sourced, double-entry** book (the UMA / overlay-manager pattern).



Invariants (continuously assertable): (1) POSITIONS: $\sum_{\text{sleeves}} \text{sleeve_qty}(\text{symbol}) == \text{broker_qty}(\text{symbol}) \forall \text{symbol}$ (2) CASH: $\sum_{\text{sleeves}} \text{sleeve_cash} == \text{broker_cash}$ (3) SLEEVE: $\text{sleeve.equity} = \text{sleeve.cash} + \sum(\text{lot market value})$

Double-entry means every share/cash movement has matched legs summing to zero, so conservation is a **checksum** — drift is detectable, never silent. Projections (sleeve positions, cash, P&L, account aggregate, holding history) are **pure folds** of the log and cannot diverge; crash recovery is a replay.

The two one-directional, self-healing sync surfaces



Event taxonomy

Capital	FundsDeposited/Withdrawn, CapitalAllocated, CapitalTransferred
Trading	OrderIntended/Submitted/Accepted/Rejected/Filled/Cancelled
Positions	LotOpened/Increased/Reduced/Closed (+ realized P&L)
Cash	CashDebited/Credited, DividendReceived, FeeCharged
Corporate	SplitApplied, SymbolChanged, MergerApplied
Recon	ExternalTradeDetected, DriftCorrected, ReconciliationAdjusted
Sleeve	SleeveOpened/Closed/Frozen/Resumed

Shadow reconciliation (by cause)

- **Our fill** not yet booked → match by `client_order_id` → sleeve. Deterministic.
- **External trade** (unknown id) → `ExternalTradeDetected` into **Unmanaged** + alert; never a strategy.
- **Corporate action** → apply lot-by-lot from activities feed, preserve provenance.
- **Material drift** → `SleeveFrozen` + alert — don't trade on a book you can't trust.

Projections — read models folded from the log

Sleeve positions / lots	open lots per sleeve with cost basis — the source of truth for attribution
Sleeve cash	<code>balance</code> · <code>reserved</code> (open buys) · <code>free = balance - reserved</code> · settled vs unsettled
Sleeve P&L	realized (from lot closes) + unrealized (lots × current price), per sleeve & strategy
Account aggregate	Σ across sleeves — the value reconciled against broker truth
Holding history	per-symbol lot open/close timeline with source labels (the "who traded this" view)

Cash mechanics — one broker pool, per-sleeve virtual balances

- **Allocate** (`Unallocated` → `strategy sleeve`) — emits `CapitalAllocated` after checking `Unallocated.free ≥ X` . Purely virtual; no broker movement.
- **Reserve** → **debit** → **release** — a buy **reserves** est. cost on submit, converts it to a real **debit** (`qty×price + fees`) on fill, and **releases** it on cancel/reject; sells credit the sleeve.
- **Transfer** (`sleeve` → `sleeve`) — virtual if free cash covers it; if cash is tied up in positions the coordinator **raises cash** (sells in the source sleeve) first.
- **Deposit / withdraw** — deposits land in `Unallocated` ; withdrawals draw free cash, liquidating if short.
- **Dividends / fees** — attributed to the lot owner; a dividend on a **shared** symbol splits **pro-rata by lot quantity** across the sleeves holding it.

Settlement is a real fork: cash accounts track `settled` vs `unsettled` (T+1) to respect good-faith / free-ride rules; margin accounts rotate same-day on buying power.

Worked example — provenance after a regime flip

One broker position per symbol, but the ledger keeps every lot's origin. After Strategy B rotates risk-off into TLT that Strategy A already holds, the user-facing holding history stays a faithful audit trail:

HOLDING HISTORY — TLT · TOTAL 327.1 SHARES · ONE BROKER POSITION, THREE ATTRIBUTED LOTS

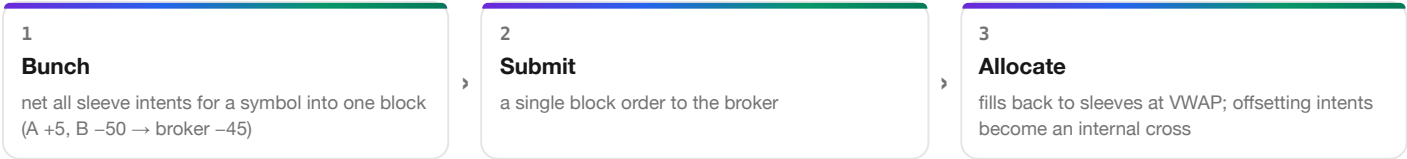
TLT					
└ Strategy A	126.32 sh	bought Jan 2	@ \$95.00	<i>(initial all-weather)</i>	
└ Strategy A	14.27 sh	bought Apr 1	@ \$100.00	<i>(quarterly rebalance add)</i>	
└ Strategy B	186.50 sh	bought Apr 1	@ \$100.00	<i>(regime flip → risk-off)</i>	

SIZING AGAINST THE SLEEVE — WHY STRATEGIES DON'T COLLIDE

An intended order is the delta between target and the sleeve's *own* holdings: `target_value = sleeve_equity × (weight/100)` ; sells reduce that sleeve's lots (FIFO). Sizing against **sleeve equity, not account equity** is exactly what lets many strategies share one account without double-counting capital. A drift tolerance (e.g. 5%) avoids churn.

Block-and-allocate execution

When multiple sleeves trade the same symbol in one cycle, the coordinator doesn't fire competing orders — it follows the institutional **block-and-allocate** pattern:

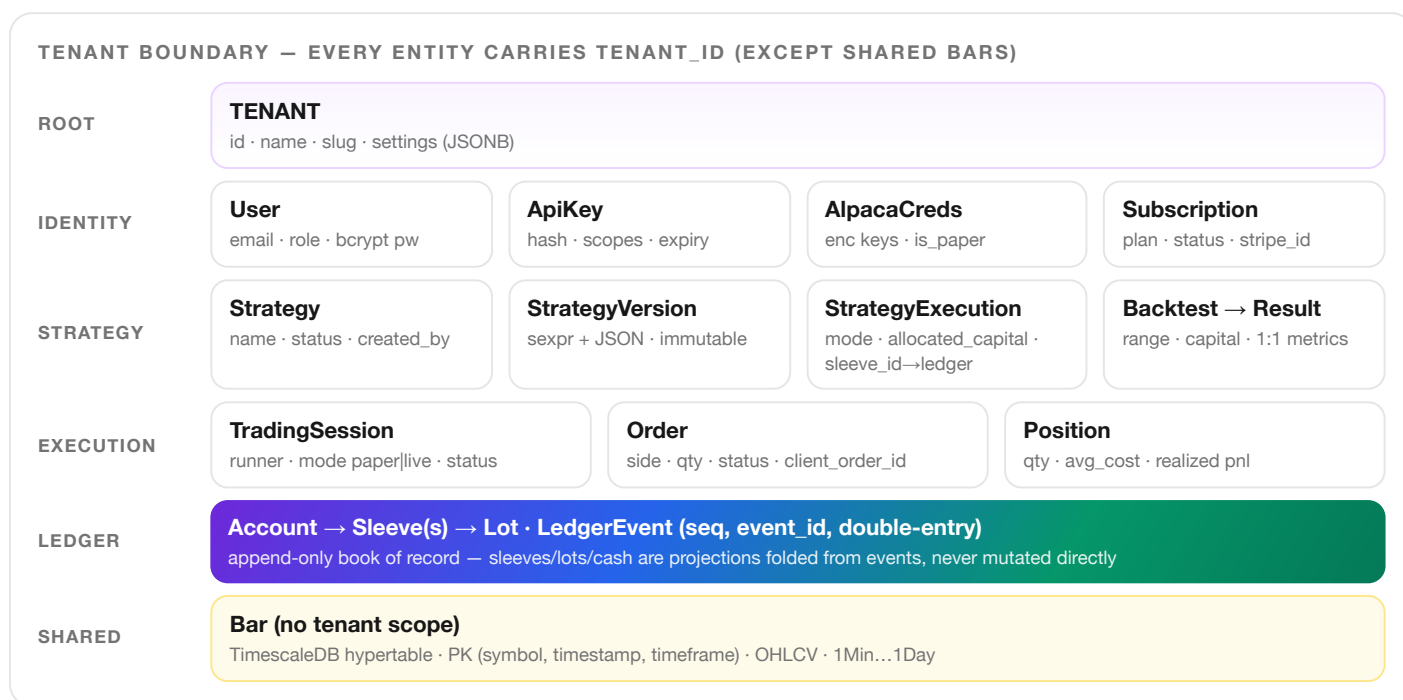


Gross execution (separate orders) makes attribution trivially 1 order ↔ 1 sleeve; netting / internal-cross adds proportional fill-splitting — where the subtle accounting bugs live — so it graduates from gross only once the ledger is proven.

Ownership: Portfolio (:8860) owns the ledger, allocation brain, projections, and reconciliation (book of record). Trading (:8850) is the execution arm — risk checks, block orders, fills. Fills flow Trading → Portfolio; intended orders flow Portfolio → Trading.

07 Data Model

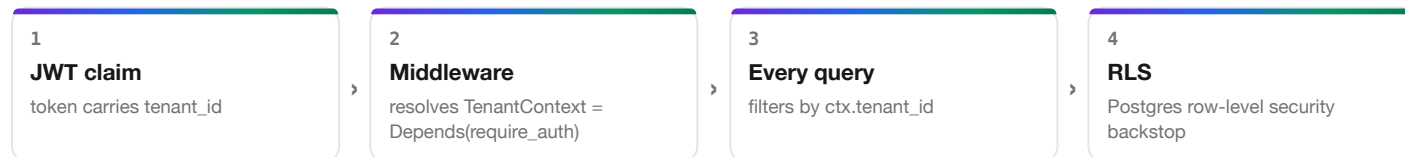
PostgreSQL with SQLAlchemy 2.0 (async). Every tenant-scoped model carries `tenant_id → tenants.id`; price `bars` are shared (TimescaleDB, no tenant scope). The ledger cluster is owned by Portfolio.



A Tenant owns Users, API keys, Alpaca creds, a Subscription, Strategies, and Audit logs. Each Strategy owns immutable Versions and spawns Executions (funded with a ledger sleeve, running as TradingSessions → Orders/Positions) and Backtests (→ 1:1 Result). Models live in `libs/db`; migrations via Alembic.

08 Multi-Tenancy & Security

Tenant isolation — defense in depth



ROW-LEVEL SECURITY — DATABASE BACKSTOP

```
ALTER TABLE strategies ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON strategies
  FOR ALL USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

Service-to-service calls propagate tenant via `X-Tenant-ID`. Missing rows return `NOT_FOUND` — never leak existence. **When in doubt, add tenant filtering.** Tenant-isolation tests assert no cross-tenant reads.

Authentication

JWT access tokens (30 min, in-memory) + refresh tokens (7 days, HttpOnly cookie). API keys

`lt_<prefix>_<random>` store only a SHA-256 hash; passwords bcrypt-hashed.

ACCESS TOKEN PAYLOAD

```
{ "sub": "user-uuid", "tenant_id": "tenant-uuid",
  "roles": ["admin"], "exp": 1705314000,
  "type": "access" }
```

RBAC & data protection

ROLE	PERMISSIONS
owner	full access, billing, users
admin	everything except billing
user	CRUD own strategies, view portfolio
viewer	read-only

Alpaca creds encrypted at rest (AES-256-GCM); TLS everywhere; secrets in GCP Secret Manager. Audit log records `user.login`, `api_key.created`, `strategy.deployed`, `order.submitted`, per tenant.

Durability & State Authority

Durability is **layered by authority** — each kind of state is owned by the system that is actually its source of truth, rather than one session-level event log. This avoids keeping multiple authorities in sync.

STATE	AUTHORITY	MECHANISM
Live order status & positions	Broker (Alpaca)	trade-updates stream + periodic reconciliation; fills are truth
Money, lots, cost basis, per-strategy P&L, provenance	Portfolio ledger	account-grain, append-only, double-entry event log
Order intent / exactly-once submission	Trading order executor	durable order rows keyed by a deterministic <code>client_order_id</code>
Strategy runtime (indicator windows, rebalance clock)	Runner	rehydrated on restart from market data + broker/ledger (checkpoint, not replay)

Idempotent submission & crash recovery

Orders use a deterministic `client_order_id` derived from the signal, so the same signal always yields the same id and Alpaca treats it as an idempotency key — a retry after a crash collapses onto the same broker order instead of placing a second one.

DETERMINISTIC CLIENT_ORDER_ID (ALSO THE LEDGER ATTRIBUTION KEY)

```
data = f"{session_id}:{symbol}:{side}:{signal_timestamp.isoformat()}"
client_order_id = "lt-" + hashlib.sha256(data.encode()).hexdigest()[16]
```

On restart the runner re-syncs equity, reconciles positions against the broker (source of truth), and re-warms indicators from market data. An order recorded but not persisted before a crash is rejected as a duplicate at the broker instead of double-filling.

RETIRED

The trading service previously carried a separate session-level event-sourcing subsystem (`trading_events` , `EventSourcedOrderExecutor` , `SessionState` replay). It was never wired into the live path and has been **retired** — the portfolio ledger is the event-sourced book of record.

10 Event Fabric

Real-time and async delivery runs on **Redis Streams** (pub/sub fully retired). The new `libs/events` wraps a **proto envelope + pluggable transport + a typed catalog**, so producers/consumers swap transports (Redis Streams today, Kafka later) without changes.

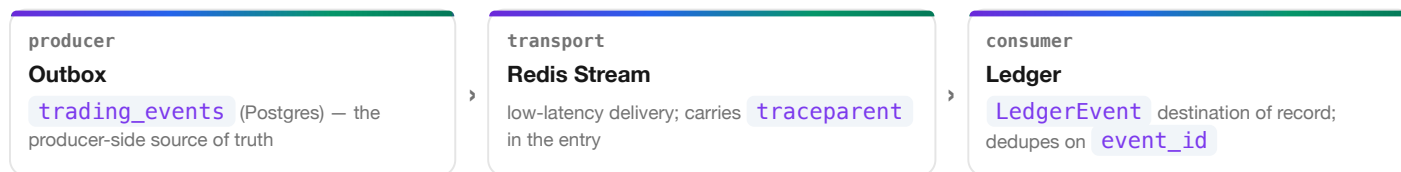
Two delivery modes

- **Tail read** — live UI/gRPC clients read from a cursor (`XREAD`); each sees the full stream and a reconnect **replays the gap** (no lost fills).
- **Consumer group** — durable competing consumers (ledger ingestion) via `XREADGROUP` + `XACK` + `XAUTOCLAIM` ; each group gets every message once, unacked entries reclaimable on restart.

Idempotency & outbox

- Stable `Event.id` ; ledger dedups on `event_id = sha256(client_order_id)` .
- At-least-once delivery + dedupe = **effectively exactly-once**.
- Trading's `trading_events` table is the producer outbox; a periodic sweep backstops any stream loss — self-healing.

Outbox → stream → ledger (self-healing)



A periodic **outbox sweep** compares `trading_events` against ledger-applied events and re-emits anything the stream dropped — so the transport can fail or restart without ever losing a fill. Recovery is convergence, not a manual replay.

CHANNELS (LOGICAL-PREFIX BOUNDED CARDINALITY)

- `lt:trading:account:{account_id}` order + position updates — tailed by UI, consumed by ledger ingestion (group `portfolio-ledger`)
- `lt:backtest:progress:{backtest_id}` backtest progress events — tailed by the watching UI

`libs/events` layout: proto `EventEnvelope` codec · `EventTransport` Protocol (redis_streams / kafka) · typed catalog (orders, positions, fills, progress, bars) · `StreamConsumer` (group, ack, retry, DLQ, lag) · `StreamFanout` (one stream → N client queues).

11 Observability

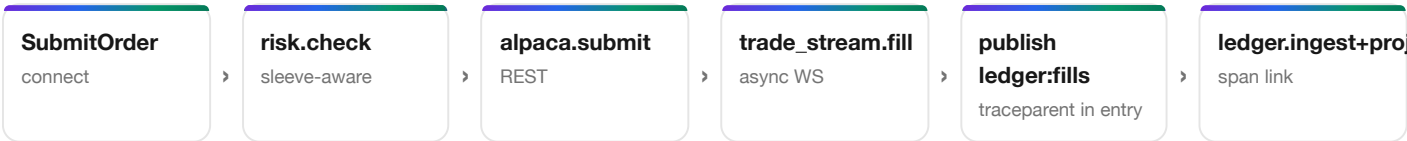
One library, one setup call: `init_telemetry(app, service=...)` wires JSON logs, a `/metrics` endpoint, RED middleware, runtime collectors, and W3C trace propagation. **OTel-native, Prometheus-exposed.**

SIGNAL	BACKEND	CARDINALITY	TENANT/SESSION?	ANSWERS
Metrics	Prometheus (via OTel)	low	no (labels); <code>plan</code> yes	"fleet p99 order latency? error rate up?"
Logs	JSON → Loki / Cloud Logging	high	yes: request/tenant/user/trace	"everything that happened for tenant X"
Traces	OTLP → Tempo / Cloud Trace	high	yes (span attrs)	"why was <i>this</i> order slow end-to-end"

CARDINALITY IS SACRED

Forbidden as metric labels: `tenant_id`, `session_id`, `user_id`, `order_id`, `client_order_id`, `symbol`, `backtest_id`, `request_id`, raw URLs, emails. They live on logs/traces. `plan` (`free/starter/pro`) is **the** per-segment business label. Per-tenant numbers come from logs/traces or bounded top-N gauges — never a label.

The money path — one trace across services



Key SLOs

SLO	TARGET
Service availability	99.9%
Order submission p99	< 1s
Order success (ex-reject)	> 99%
Market-data freshness p95	< 2s
Ledger reconcile freshness	< 5 min
Backtest completion p95	< 120s

Naming & layout

`llamatrade_<domain>_<noun>_<unit>` with a validated label allow-list. Typed domain namespaces: `metrics.trading.*`, `metrics.ledger.*`, `metrics.marketdata.*`, ... The crown jewels are Trading (submission/fill latency, slippage, circuit breaker) and Ledger (events ingested, reconciliation drift, `ledger_vs_broker_mismatch_dollars` ≈ 0).

Graceful degradation: no OTLP collector → tracing no-ops; no `prometheus_client` → metrics no-op. Telemetry never crashes a request.

12 Stack & Deployment

Technology

- **Backend:** Python 3.14, FastAPI hosting Connect/gRPC ASGI; Celery + Redis for jobs.
- **Data:** PostgreSQL 16 + TimescaleDB (bars); Redis 7 (cache, streams).
- **Frontend:** React 18 + Vite, TypeScript, Tailwind, Zustand, Connect client, canvas builder.
- **Libs:** alpaca · proto · db · dsl · compiler · common · telemetry · events.

Runtime

- ≥ 2 replicas/service, CPU/mem requests+limits, gRPC liveness probes, HPA (CPU 70%).
- Every service exposes `/health` → `{status, service, version}`.

Infrastructure (GCP)

- GKE Autopilot · Cloud SQL (Postgres/Timescale) · Memorystore (Redis).
- Cloud Storage · Cloud CDN · L7 LB (TLS termination) · Secret Manager.
- Cloud Monitoring/Logging; CI/CD via GitHub Actions + Cloud Build.

Environments

- **Staging** — auto-deploy on merge to `main`.
- **Production** — tag + manual approval (`make deploy-prod`).
- Backups: daily DB snapshots (7-day), Redis persistence, versioned object storage; Terraform-reproducible.

WHERE TO GO DEEPER

This reference summarizes the canonical docs in `.docs/`: `architecture.md` (spine), `portfolio-ledger.md`, `strategy-dsl.md`, `telemetry.md`, the per-service docs under `services/`, and the `decisions/` and `planning/` folders. Proto contracts in `libs/proto` are authoritative for every RPC.